Paw Tracker

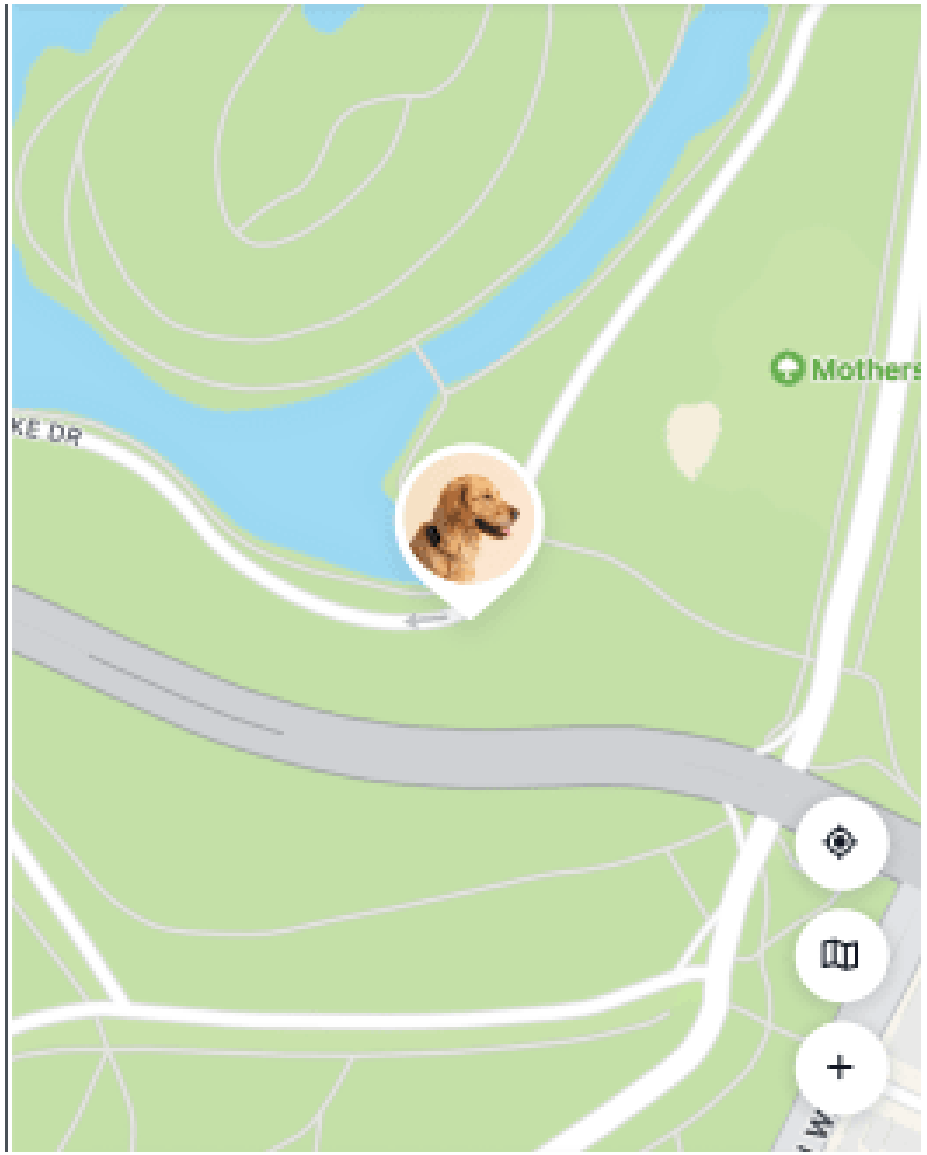
Dog Sport Tracker App

Mobile Application Development Project

Emmi Korhonen & Farnaz Jamilpanah

Sprint 1 Goals:

- Create folder structure
- Integrate GPS
- Integrate Map API
- Write README
- Write Sprint1 Review and Sprint2 Backlog



2 25 Mar – 1 Apr (5 work items)

og owner, I want the app to track my walk using GPS so that I ... TO DO

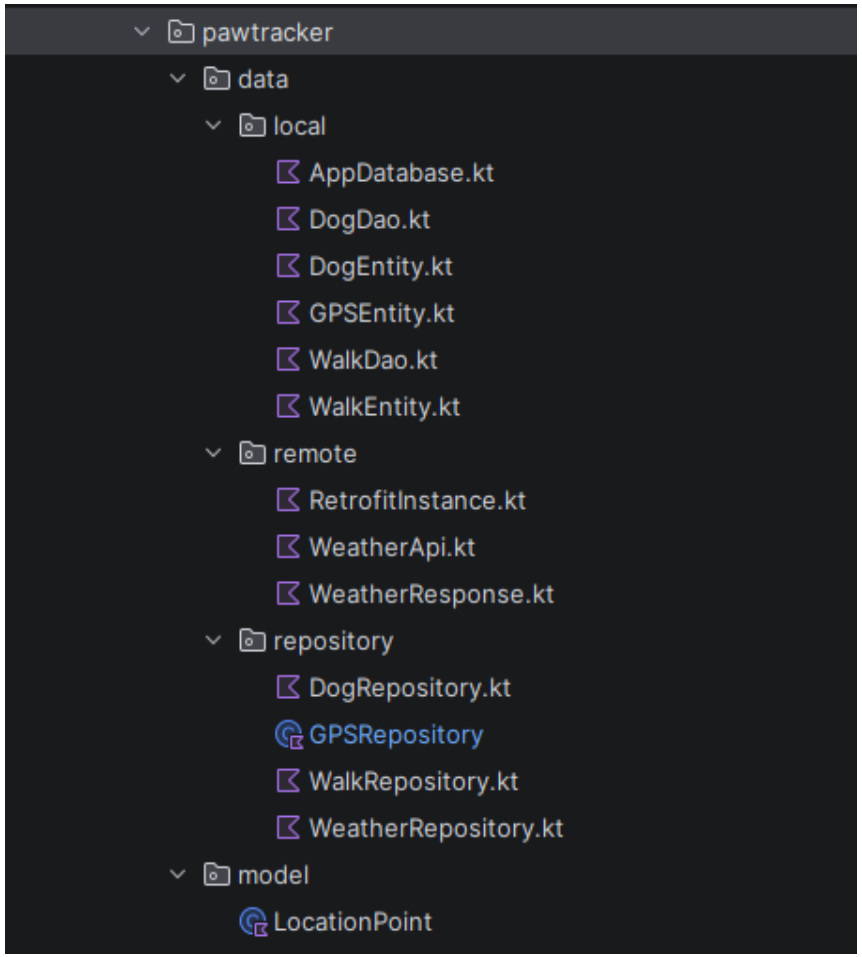
acking + Eepos DONE

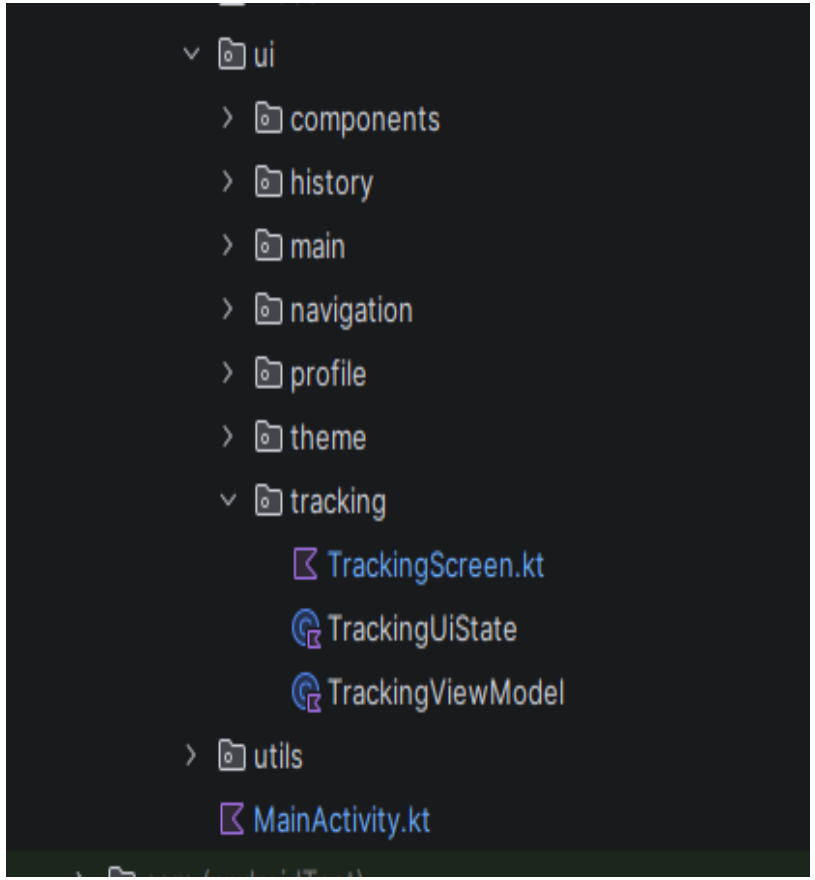
ew DONE

ME DONE

folder structure DONE

Folder Structure:





What I (Farnaz) Worked On:

GPS integrating

Add the dependency (build.gradle.kts)

```
androidTestImplementation(libs.androidx.compose.ui.test.junit4)
debugImplementation(libs.androidx.compose.ui.tooling)
debugImplementation(libs.androidx.compose.ui.test.manifest)

implementation("com.google.android.gms:play-services-location:21.3.0")
```

*Permission handling:*Add inside <manifest.xml>:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

GPSRepository:

using FusedLocationProvider

```
hasLocationPermission()
setupLocationRequest()
startLocationUpdates()
LocationCallback()
GetLastLocation()
stopLocationUpdates()
```

```
GPSRepository(
rivate val context: Context

Usages
rivate val fusedLocationProviderClient =
    LocationServices.getFusedLocationProviderCli

Usages
rivate lateinit var locationRequest: LocationRe
Usages
rivate lateinit var locationCallback: LocationC
```

TrackingViewModel:

```
class TrackingViewModel(
    private val gpsRepository: GPSRepository
) : ViewModel() {
```

```
fun stopTracking()
```

```
fun loadLastLocation()
```

```
/**
 * Start tracking either real GPS or mock location
 */
1 Usage
fun startTracking() {
    _uiState.update { it.copy(tracking = true) }

    if (useMockLocation) {
        viewModelScope.launch {
            mockLocationFlow().collect { point ->
                _uiState.update { state ->
                    state.copy(
                        currentLocation = point,
                        points = state.points + point
                    )
                }
            }
        }
    } else {
        gpsRepository.startLocationUpdates { point ->
            viewModelScope.launch {
                _uiState.update { state ->
                    state.copy(
                        currentLocation = point,
                        points = state.points + point
                    )
                }
```

Emmi tasks:

Google Map integrating

Add Google Maps Compose dependency (build.gradle.kts)
Add Google Maps API key (AndroidManifest.xml)
Add strings (`strings.xml`)

TrackingScreen basic layout, UiState, ViewModel with mock data, primary theme, GitHub README

```
<resources>
    <string name="app_name">Paw Tracker</string>

    <string name="tracking_distance">Distance</string>
    <string name="tracking_time">Time</string>
    <string name="tracking_start">Start</string>
    <string name="tracking_stop">Stop</string>
</resources>
```

```
implementation("com.google.maps.android:maps-compose:4.3.3")
implementation("com.google.android.gms:play-services-maps:18.2.0")
```

```
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.material3:material3")
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
```

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="${MAPS_API_KEY}" />
```

TrackingScreen:

```
/**
 * Main TrackingScreen composable
 */
2 Usages
@Composable
fun TrackingScreen(viewModel: TrackingViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsState()

    TrackingLayout(
        uiState = uiState,
        onStart = { viewModel.startTracking() },
        onStop = { viewModel.stopTracking() }
    )
}
```

```
fun TrackingLayout(
```

```
fun TrackingMap()
```

```
fun TrackingStatistics()
```

```
fun ControlButtons()
```

Markers, Camera, routes, Polyline

```
val cameraPositionState = rememberCameraPositionState {
    position = CameraPosition.fromLatLngZoom(
        target = com.google.android.gms.maps.
        model.LatLng( latitude = 60.1699,  longitude = 24.9384),
        zoom = 15f
    )
}
```

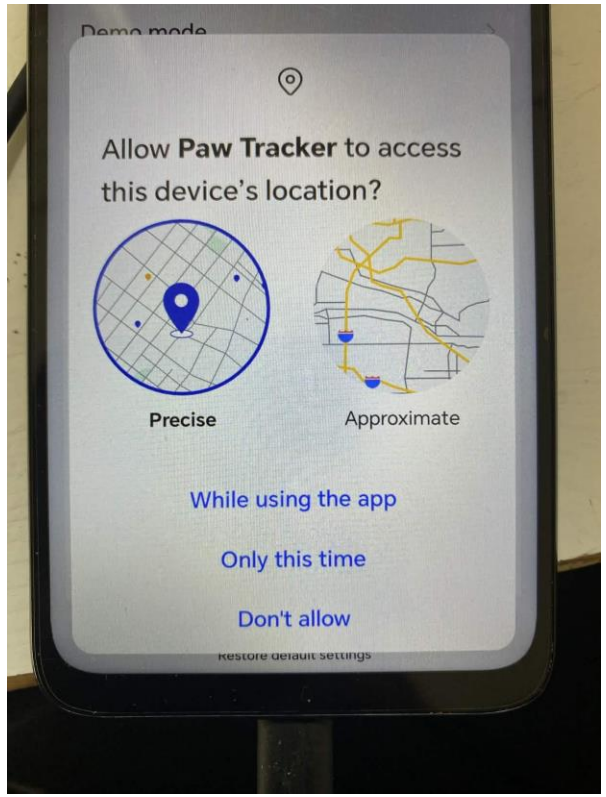
```
uiState.currentLocation?.let { point ->
    Marker(
        state = MarkerState(
            position = com.google.android.gms.maps.
            model.LatLng( latitude = point.lat,  longitude = point.long)
        ),
        title = "Current"
    )
```

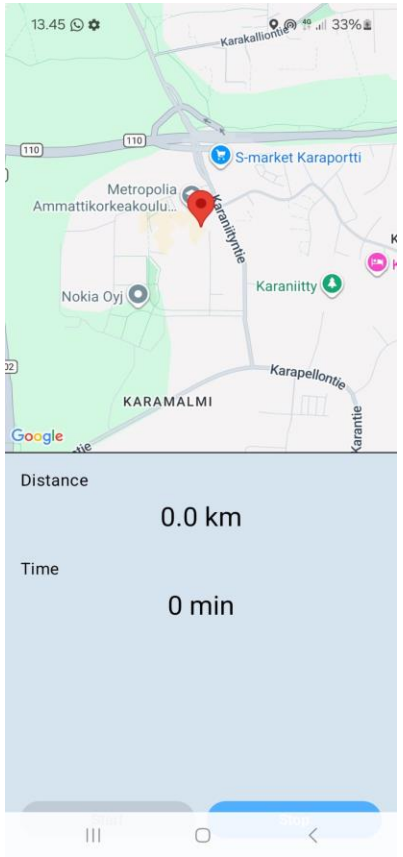
```
Polyline(
    points = uiState.points.map
    { com.google.android.gms.maps
        .model.LatLng( latitude = it.lat,  longitude = it.long) }
)
```

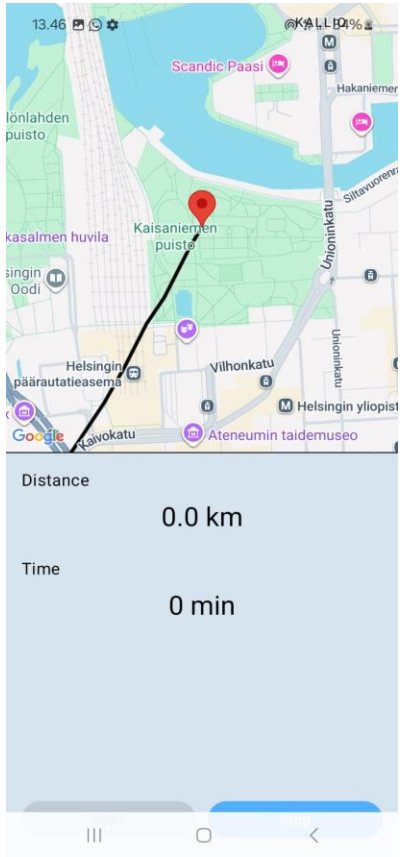
MainActivity:

```
class MainActivity : ComponentActivity() {
    1 Usage
    private val trackingViewModel by lazy { TrackingViewModel(gpsRepository) }

    // Permission launcher must be inside class
    1 Usage
    private val requestPermissionLauncher =
        registerForActivityResult( contract = ActivityResultContracts.RequestPermission()) { granted ->
            // You can show a message if needed
        }
    // Permission request function
    1 Usage
    private fun requestLocationPermission() {
        requestPermissionLauncher.launch( input = android.Manifest.permission.ACCESS_FINE_LOCATION)
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Ask for permission
        requestLocationPermission()

        // Edge to edge
        enableEdgeToEdge()

        //  Set content
        setContent {
            PawTrackerTheme {
                TrackingScreen(viewModel = trackingViewModel)
```

Application Demo







README

PawTracker – Dog Activity Tracker

PawTracker is an Android application designed to help dog owners monitor and improve their dog's daily physical activity.

Dog owners often don't know whether their pets are getting enough exercise, which can negatively affect their health. PawTracker provides a simple, visual interface that displays dog's walking routes.

Application uses the phone's GPS to automatically record routes, calculate distance and duration, and store the data for long-term tracking.

Core Features

- **GPS Tracking** – Records walking routes in real time
- **Map View** – Visualizes routes using Google Maps
- **Activity Statistics** – Distance, duration and daily/weekly totals
- **History** – Stores past walks using Room database
- **Modern UI** – Clear and responsive design
- **Unit Testing** – Tests for ViewModel logic and data handling

Optional Features

- Dog profile with name, breed and profile picture
- Dog breed API for recommended activity levels
- Health log for tracking observations for wellbeing
- Weather integration
- Activity reminder notifications
- AI-based weekly activity analysis

Architecture

- MVVM
- StateFlow

Technology Stack

- **Language:** Kotlin
- **UI:** Jetpack Compose
- **Database:** Room
- **Location Services:** GPS
- **Maps:** Google Maps API

Testing

- JUnit
- UI testing

Tools

- **IDE:** Android Studio
- **Version Control:** GitHub

Retrospective

Challenges:

- Uses real GPS (FusedLocationProviderClient)
- Uses mock location for testing the map
- GPS not updating on emulator
- Google Maps API key setup

What went well:

- Team work
- We successfully installed and used Map and Gps together

Sprint 2 backlog:

- Distance and Time
- Room database
- HistoryScreen
- Save walk data

Thank you and questions